Algorithms

Lecture 8

Algorithms

- The Concept of an Algorithm
- Algorithm Representation
- Algorithm Discovery
- Iterative Structures
- Recursive Structures
- Efficiency and Correctness

Algorithms We Have Learned So Far

- Converting numbers to their different numerical representations
- Computing arithmetic and logical operations on numbers
- Detecting/correcting errors
- Compressing/decompressing data
- Controlling parallel operations using semaphore
- Euclidean algorithm, Babylonian algorithm, etc
- Some algorithms written using machine language

Formal Definition of Algorithm

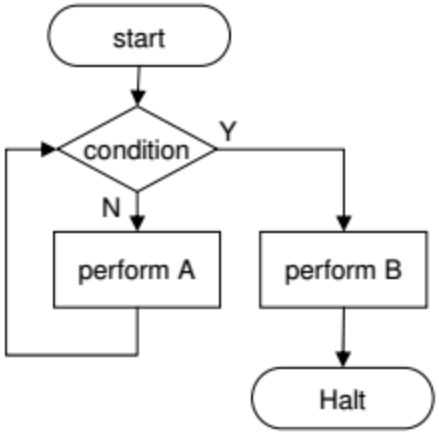
- An algorithm is an **ordered** set of **unambiguous**, **executable** steps that defines a **terminating** process.
  - "Ordered" does not imply "sequential" -> there are parallel algorithms. "Ordered" implies that there is a concrete plan of the execution, be it in single thread, or multiple threads.
  - "Unambiguous" means that each step must uniquely and completely specify the set of actions required to undertake without involving any creativity.
  - "Executable" means that each step could be accomplished by the executor (typically computer)
  - "Terminating" - any algorithm should reach its end.

Terminology Recap

- What is the difference between an algorithm, a program and a process?
  - Program is a formal implementation *(typically using computer programming languages)* of an algorithm
  - Process is the activity of executing an algorithm (or equivalently, a program)

Algorithm Representation

- An algorithm is an abstract concept
  - There can be several physical implementations of an algorithm (using same or different programming language)
  - Given same input, different implementations of an algorithm should produce the same output
  - For example, an algorithm is like a story, and an implementation is like a book on the story.
- So, we must find a way to separate an algorithm from the actual implementation, but still precisely define it.
- In 1950s-1970s, flowcharts were popular way of describing algorithms, however they get messy if the algorithm is complex.
- Nowadays, *pseudocode (human-readable programming language)* is popular among computer scientists

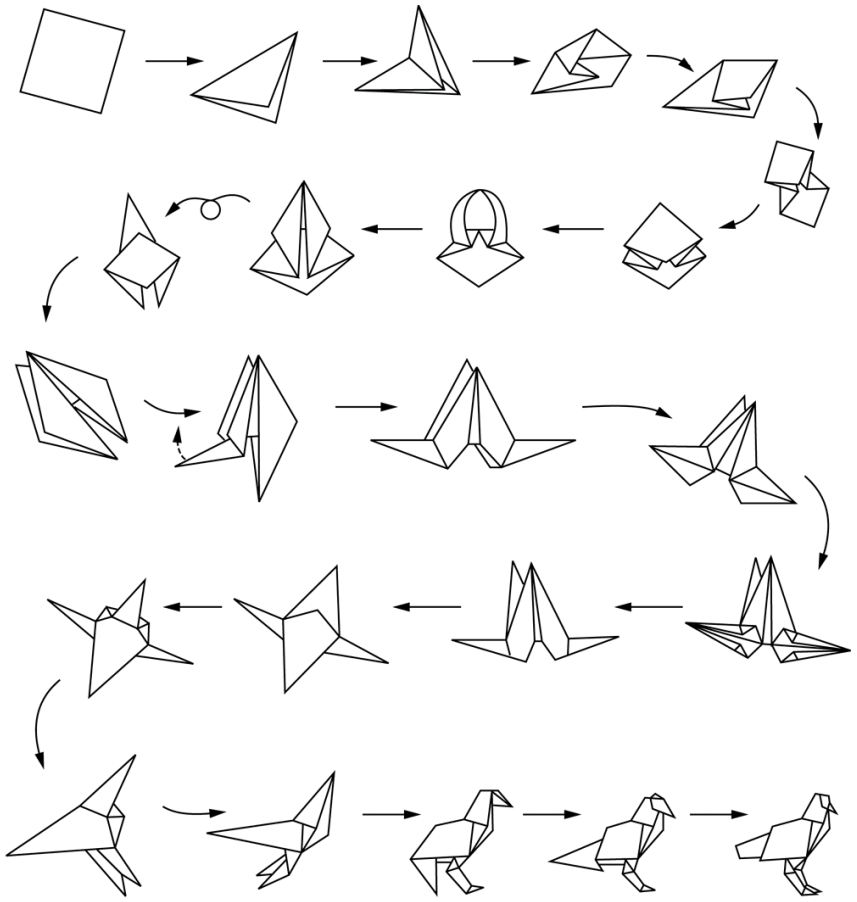


Algorithm Representation

- Technically, any language (e.g. English) can be used as a pseudocode language to describe an algorithm
- In practice, a good pseudocode language must avoid any ambiguities:
  - A language with a small set of well-defined building blocks (aka *primitives*) can remove any ambiguity
  - Each primitive is a single operation you can apply to the data

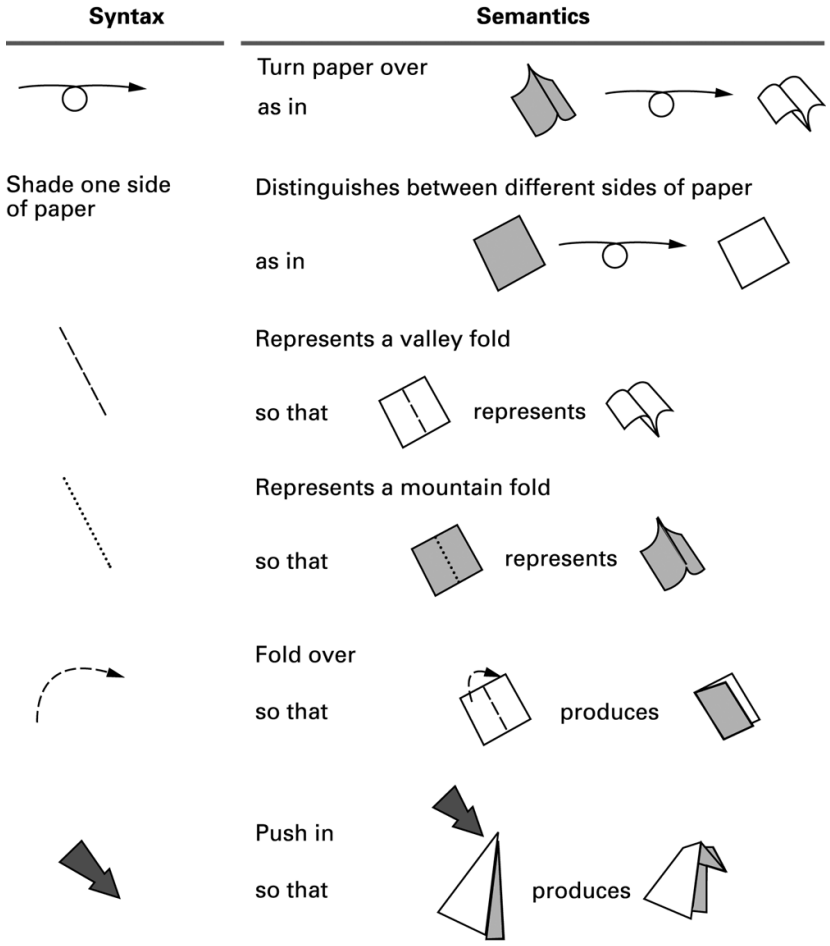
Example Origami

- Origami, itself, is a very complicated procedure
- However we can determine several operations as our *primitives,* and reference them when needed.
- What actions can be denoted as primitives in origami?



Origami primitives

- Each primitive has its **syntax** and **semantics:**
  - Syntax refers to primitive's symbolic notation
  - Semantics refers to primitive's meaning
- Once, we have these instructions defined, we can use them to explain sequence of actions unambiguously, so one can build an origami.
- However, primitives defined in our programming language also limit our implementation capacity
- What can't we do with these primitives?



Pseudocode (Python) Primitives

- Assignment

```
name = expression
```

- Example

```
RemainingFunds = CheckingBalance +
                                 SavingsBalance
```

Pseudocode Primitives (continued)

- Conditional selection

```
if (condition):
    activity
```

- Example

```
if (sales have decreased):
    lower the price by 5%
```

Pseudocode Primitives (continued)

- Conditional selection

```
if (condition):
    activity
else:
    activity
```

- Example

```
if (year is leap year):
    daily total = total / 366
else:
    daily total = total / 365
```

Pseudocode Primitives (continued)

- Repeated execution

```
while (condition):
    body
```

- Example

```
while (tickets remain to be sold):
    sell a ticket
```

Pseudocode Primitives (continued)

- Indentation shows **nested** conditions

```
if (not raining):
    if (temperature == hot):
        go swimming
    else:
        play golf
else:
    watch television
```

Pseudocode Primitives (continued)

- Define a function

```
def name():
```

- Example

```
def ProcessLoan():
```

- Executing a function

```
if (. . .):
    ProcessLoan()
else:
    RejectApplication()
```

The procedure Greetings

```
def Greetings():
    Count = 3
    while (Count > 0):
        print('Hello')
        Count = Count - 1
```

Pseudocode Primitives (continued)

- Using parameters

```
def Sort(List):
    .
    .
```

- Executing Sort on different lists

```
Sort(the membership list)
Sort(the wedding guest list)
```

Some Algorithms That We Know

Euclidean Algorithm

```
def gcd(m,n):
    while(n):
        m,n=n,m%n
    return m
```

Babylonian Algorithm

```
def root(n, guess=n/2, precision=0.001):
    while True:
        if abs(guess**2-n)<=precision):
            return guess;
        guess=(guess+n/guess)/2
```

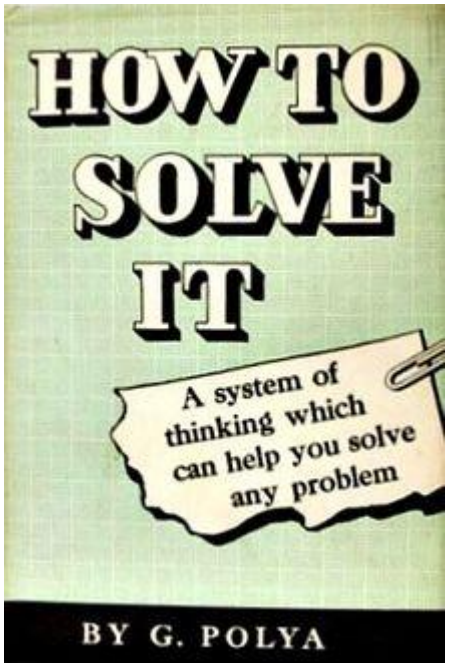
Algorithm Discovery

- Techniques to discover an algorithm to solve real-world problem often requires specific **domain knowledge** that we won't learn in Computer Science class ☹.
- However, solving real-world problems with complex algorithms would usually require the knowledge of small "standard algorithms" which we do learn in Computer Science class.
  - For example, a good algorithm for "sorting numbers" could be a very handy tool in solving multiplicity of other bigger problems.

Polya's Problem Solving Phases

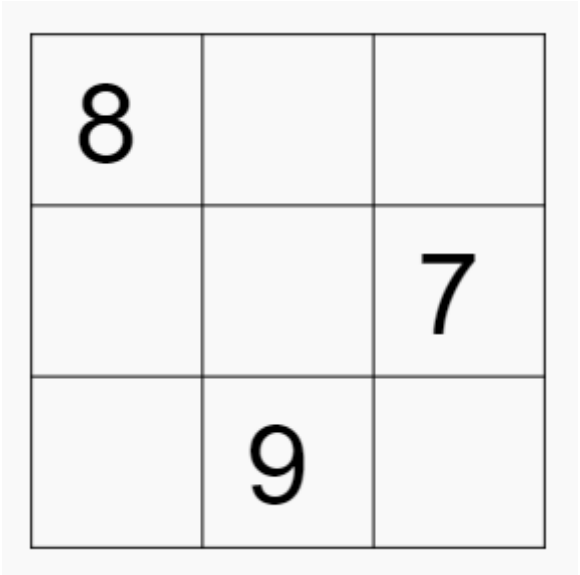
- 1. Understand the problem.
  - What are you asked to find/show?
  - Can you restate the problem in your own words?
  - Can you think of a picture or a diagram that might help you understand it?
- 2. Devise a plan for solving the problem.
  - Extend existing knowledge
  - Eliminate possibilities
  - Relax the problem
- 3. Carry out the plan.
- 4. Evaluate the solution for accuracy and its potential as a tool for solving other problems.

Book published in 1945 by George Polya.



Bruteforce Search

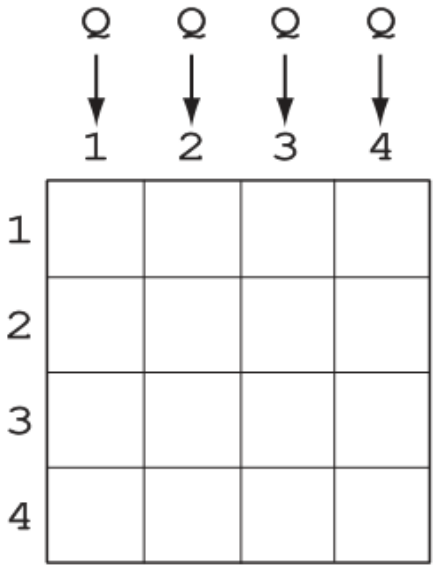
- Try all possible candidate solutions
- **Magic Square**
  - Fill the 3 × 3 table with nine distinct integers from 1 to 9 so that the sum of the numbers in each row, column, and corner-to-corner diagonal is the same



Backtracking

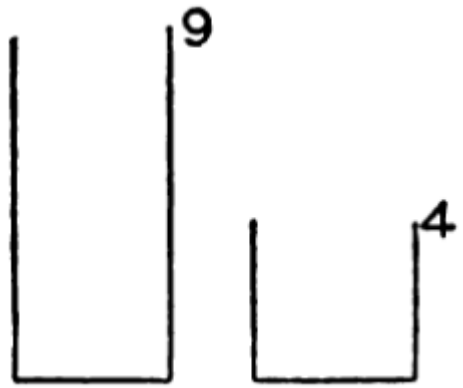
- Continue legitimate moves until hit the hopeless dead end.
- Cancel previous move, pick the other alternative move and continue in similar fashion.

4 queens problem



Working Backwards

- How can you take from the river exactly 6 litres of water using 9- and 4-litre buckets?



Extend Existing Knowledge

- Before A, B, C and D ran a race they made the following predictions:
  - A predicted that B would win
  - B predicted that D would be last
  - C predicted that A would be third
  - D predicted that A's prediction would be correct
- Only one of these predictions was true, and this was made by the winner.
- In what order did A, B, C and D finish the race?

Eliminate Possibilities

- Person A is charged with the task of determining the ages of B's three children.
  - B tells A that the product of the children's ages is 36.
  - A replies that another clue is required.
  - B tells A the sum of the children's ages.
  - A replies that another clue is needed.
  - B tells A that the oldest child plays the piano.
  - A tells B the ages of the three children.
- How old are the three children?

a. Triples whose product is 36

(1,1,36)	(1,6,6)
(1,2,18)	(2,2,9)
(1,3,12)	(2,3,6)
(1,4,9)	(3,3,4)

b. Sums of triples from part (a)

1 + 1 + 36 = 38	1 + 6 + 6 = 13
1 + 2 + 18 = 21	2 + 2 + 9 = 13
1 + 3 + 12 = 16	2 + 3 + 6 = 11
1 + 4 + 9 = 14	3 + 3 + 4 = 10

Relax the Problem

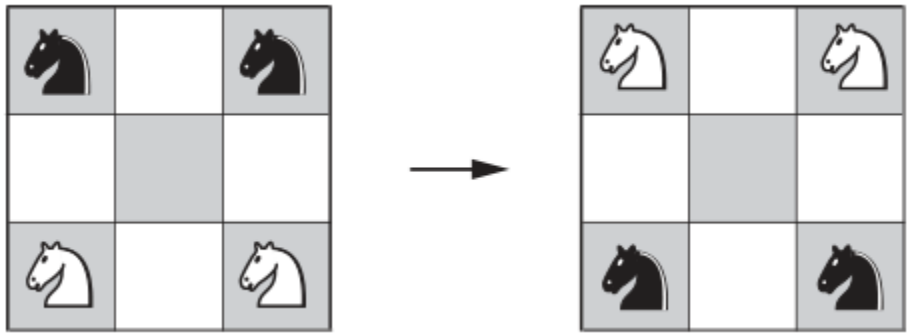
- Look for solutions of an easier, related problems or some special cases:
  - As you step from a pier into a boat, your hat falls into the water without you knowing it.
    - The river is flowing at 2.5 km/hr.
    - You begin to travel at speed of 4.75 km/hr upstream.
    - In 10 minutes, you notice you dropped you hat.
    - How long will take to catch up your hat?
- How can this problem relaxed?

Transforming the Problem

- Change the representation of the problem, and then apply your solution to it.
- **Anagram Detection:**
  - Anagrams are words that are composed of the same letters; for example, the words "*eat*," "*ate*," and "*tea*" are anagrams. Devise an algorithm to find all sets of anagrams in a large file of English words.

Another Transformation Puzzle

- The goal is to switch the knights in the minimum number of moves.



Other Techniques for Problem Solving

- Top-down approach (iterative refinement)
  - Progresses from general to specific solutions
- Bottom-up approach (recursion)
  - Progresses from specific to general solutions
- Greedy approach
  - Using heuristic function to make choice
- Dynamic Programming
  - Memoization of previously found results

Suggested Reading/Activity

- Walk through LeetCode Beginner's Guide
- Read George Polya's "How to Solve It" book
- Participate in programming challenges such as Codewars, HackerRank, Project Euler and TopCoder
- Review Norwig's Pytudes